

PropertyScope NSW

Release 1 / Technical report

Shared MCP and RAG services, grounded answers and an extended agentic loop for five integrated property-research features.

41026 Advanced Software Development

Assessment 2: Release 1, MCP, RAG and Intelligent Agent Integration

Submission	Group 20
Due	4 October 2026, 11:59 pm Sydney time
Repository	MattShelton04/41026ASDProject
Showcase video	Pending
Showcase	Week 9 class, 2 October 2026
Commit reference	Pending

TO COMPLETE / TEAM

Add the published showcase video URL (10 minutes maximum, viewable by the tutor) and the final commit SHA to the table above.
Build the final PDF with `--final --baseline <sha>`.

This report separates live-stack observations from deterministic validation and fixtures. Repository links are pinned to the commit reference above.

Contents

Select a chapter or page number to navigate. PDF bookmarks include every subsection.

1 Project overview and Release 1 scope	3
2 Functional requirements	4
3 Non-functional requirements	5
4 Release 1 architecture	5
5 MCP and RAG design	6
6 Validation and results	9
7 Integration summary	14
8 Known issues and limitations	14
9 Contributions, repository and showcase	15
Appendix A Registered tool inputs and outputs	16
Appendix B Knowledge sources	17
Appendix C Contribution logs and Release 1 commits	18
Appendix D Evidence index and reproduction	19

1 Project overview and Release 1 scope

PropertyScope NSW brings NSW property identity, recorded sales, suburb context, planning and building evidence, and a buyer's own research into one application. Release 0 delivered five independently owned feature slices behind a shared HTMX shell. Each slice has a containerised frontend, backend/API and owned database service, and uses a shared AI-mode service for bounded Plan, Act, Observe and Adapt runs.

Release 1 keeps that application and adds three shared services that run on the host, outside Docker: an MCP server that exposes each feature's registered tools, a RAG server that retrieves cited project guidance, and MCP and RAG validation modes for the shared agentic loop. Every feature reaches these services from its own frontend through its own backend and AI-mode. Answers show their sources and a confidence category, or say that the available context is insufficient.

1.1 Responsibilities

Student	Feature	Release 1 responsibility
1 Matthew Shelton	Data Platform and Property Discovery	Shared MCP server, RAG server, loop validation modes, host runtime, adoption kit; Feature 1 corpus and grounded assistant
2 Burhan Naeem	Property Sales Explorer and Market Cases	Feature 2 corpus, grounded assistant, MCP/RAG evidence and student-2.yml
3 James Huang	Suburb Crime and Liveability Analytics	Feature 3 corpus, grounded assistant, MCP/RAG evidence and student-3.yml
4 Michael White	Site Planning and Building Due Diligence	Feature 4 corpus, grounded assistant, MCP/RAG evidence and student-4.yml
5 Derek Song	Buyer Journey and Agent Workspace	Feature 5 assistant routes, corpus, MCP/RAG evidence and student-5.yml
Group	Integrated application	Compose deployment, integration validation, report and showcase video

Feature 2 also registers an argument-free capability tool describing the area and its refusals, and withholds its assistant until a case is selected, so every question is scoped to saved research.

TO COMPLETE / STUDENTS 3-5

Confirm your row, then add one sentence each on the feature updates you made in Release 1 beyond MCP/RAG adoption (if any).

2 Functional requirements

ID	Requirement	Acceptance check
FR1	One shared MCP server runs on the host and lists every enabled feature's registered tools with their JSON schemas	<code>tools/list</code> returns each feature's catalogue; Section 5.1
FR2	MCP executes a registered tool only for the feature, run, call and arguments bound in signed invocation metadata, and returns a structured result	MCP loop validation passes; mismatched metadata is rejected in tests
FR3	One shared RAG server ingests a feature's registered corpus into an immutable version and retrieves cited excerpts for that feature only	Ingest and replay keep the version; retrieval returns citations
FR4	A grounded answer shows a summary, source citations (title, excerpt, date, corpus version) and a confidence category	Frontend screenshots for every feature, Section 6.3
FR5	When no passage meets the relevance floor, or RAG is unavailable, the answer says the context is insufficient and cites nothing	Insufficient-context screenshot per feature
FR6	Each feature frontend reaches MCP and RAG only through its own backend/API and AI-mode	Architecture validator; Figure 2
FR7	The shared agentic loop keeps its Release 0 mode and adds separate MCP and RAG validation modes that write captured output	<code>dev.py ai validate mcp</code> and <code>rag</code> , Section 6.2
FR8	Release 0 Compose deploys the containerised features, with backends configured to reach the host services	Compose environment, Section 6.6
FR9	Student workflows build and test each feature with MCP and RAG disabled	Workflow runs, Section 6.5

3 Non-functional requirements

Quality	Requirement and measure	Why it matters
Security	MCP and RAG bind to loopback only. AI-mode, reachable from Docker, rejects every route except liveness without a 32 to 128 character service token (observed: 401 without, 200 with)	The host listener must not become an open model proxy
MCP tool boundaries	A run can call only its feature's allowlisted tools. Tools with side effects require human approval; Feature 1's two write tools are approval-gated	A model must not choose destinations or trigger writes
Grounding and traceability	Every guidance claim cites a retrieved passage and every current fact cites a tool call. Answers store the corpus version, and completion fails if the active version changed	A reader can check each claim against its source
Reliability	An MCP outage fails the run in a bounded time without retrying (observed 2.2 s, <code>mcp_unavailable</code>). A RAG outage yields an insufficient answer while ordinary CRUD keeps working	Failures stay visible and contained
Performance	Retrieval median 0.08 s and slowest 0.13 s in the retrieval evaluation; ingesting 19 documents takes 13.8 s on CPU	Retrieval must not dominate answer time
Retrieval quality	Expected-source recall@5 of at least 0.9 on the authored question set (measured 0.963)	Grounded answers depend on finding the right passage
Usability	Sources open with the keyboard; unsafe links are rejected; no horizontal overflow at 320 px	Evidence must be readable and safe for all users
Maintainability	A feature adopts MCP and RAG by declaring a corpus in its own <code>feature.yaml</code> , without editing shared files	Five owners can integrate without conflicts
Interoperability	MCP uses the official SDK's Streamable HTTP transport and the existing JSON Schema tool definitions	Any MCP client can discover and call the tools
Availability	MCP and RAG stop and start independently. A restart keeps the corpus version and run history (observed)	One service's outage does not remove the others

4 Release 1 architecture

Figure 1 shows the containerisation boundary. Docker Compose runs the shared edge and the five feature slices, as in Release 0. AI-mode, MCP, RAG and the loop always run as host processes, which `uv run scripts/dev.py stack up` starts before the containers. This follows the labs: the AI tier is local developer infrastructure, disabled in CI and the cloud, and it keeps model caches and credentials on the host. Backends reach AI-mode at `host.docker.internal:5005` with a service token. MCP calls a feature's tool through that feature's published loopback port, so it never opens a feature database. The architecture check fails the build if any Compose file defines an AI service (ADR-046).

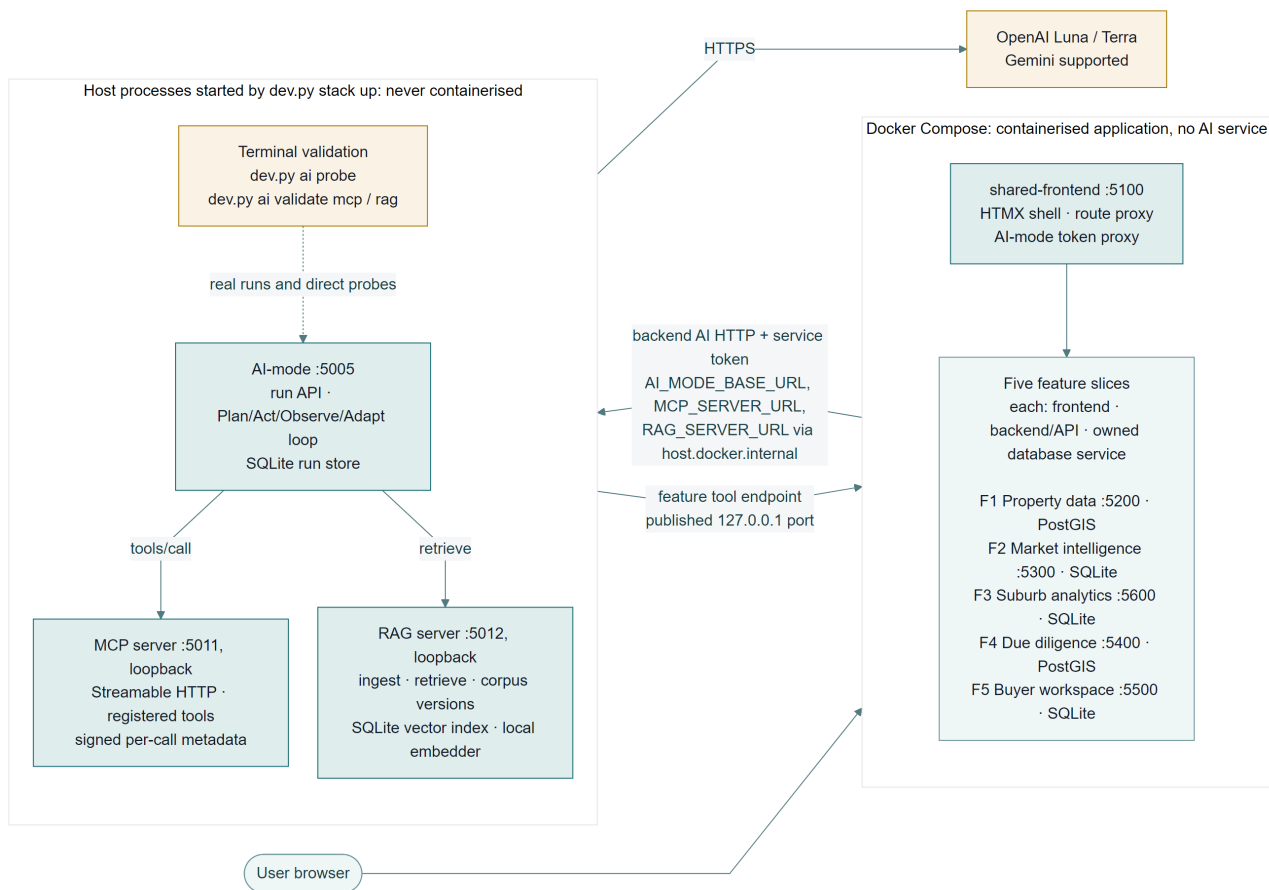


Figure 1 Release 1 architecture and containerisation boundary

4.1 Repository structure

ai-services/agent-core/	Plan/Act/Observe/Adapt runner, grounding and confidence rules
ai-services/ai-mode/	Run API, provider adapters, MCP tool executor, RAG client
ai-services/mcp-server/	Shared MCP server (Streamable HTTP) over registered feature tools
ai-services/rag-server/	Corpus ingestion, local embeddings, SQLite vector index, retrieval
shared/tool-runtime/	Catalogue parsing, bounded HTTP dispatch, signed invocation metadata
shared/frontend/ai-chat/	Shared assistant: citations, confidence, insufficient-context states
student-N/	Feature slice: frontend, backend, database, tool-catalog.yaml, feature.yaml, config/rag/ corpus (where registered)
deployment/	Enabled features and generated Compose and runtime projections
scripts/dev.py	Stack, host AI runtime and loop validation commands
.github/workflows/	integration-ci.yml and student-1.yml to student-5.yml

5 MCP and RAG design

Figure 2 follows one question through the system. The frontend sends it to its own backend, which starts an AI-mode run with the feature's tool allowlist and corpus. The planner can only choose registered tools. Tool calls go through MCP to the owning backend; retrieval goes to RAG for that feature's corpus only. Retrieved text is treated as untrusted evidence, never as instructions.

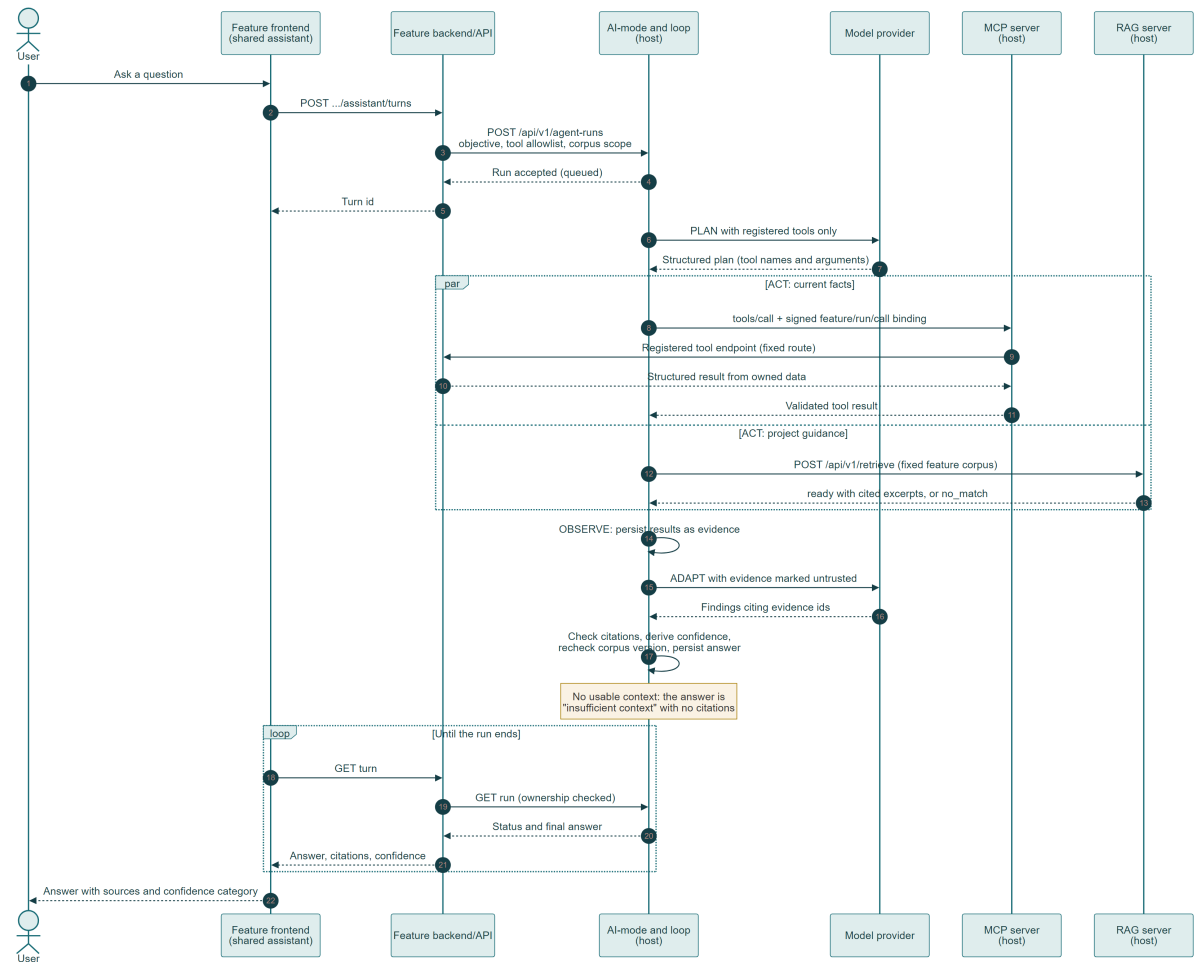


Figure 2 MCP and RAG interaction flow from a feature frontend

5.1 Registered MCP tools

Each feature owns its tool catalogue. MCP exposes the same names and schemas, and calls the same backend routes, as Release 0's direct HTTP path. AI-mode signs each call's feature, run, call id, arguments, deadline and approval state, and MCP checks that binding before dispatch. The table is generated from the checked-in catalogues when the report is built; Appendix A lists inputs and outputs.

Student	Feature	Registered MCP tools	Access	RAG corpus
1	Data Platform and Property Discovery	platform.capabilities.v1, data.sources.v1, data.releases.v1, data.runs.v1, data.release_inspect.v1, data.run_inspect.v1, data.run_explain.v1, data.release_compare.v1, data.coverage.v1, property.search.v1, property.inspect.v1, property.locality_summary.v1, data.run_retry.v1, data.release_publish.v1	12 read-only, 2 write, 2 need approval	operator-guidance (19 documents)
2	Property Sales Explorer and Market Cases	market.capabilities.v1, market.cases.inspect.v1, market.sales.summary.v1	3 read-only	operator-guidance (3 documents)
3	Suburb, Crime and Liveability Analytics	suburb.snapshot.v1, crime.compare.v1, crime.methodology.v1	3 read-only	Not registered yet

Student	Feature	Registered MCP tools	Access	RAG corpus
4	Site, Planning and Building Due Diligence	duediligence.review.inspect.v1, duediligence.evidence.summary.v1	2 read-only	Not registered yet
5	Buyer Journey Workspace	buyer.cases.inspect.v1, buyer.notes.list.v1, buyer.tasks.list.v1, buyer.evidence.collect.v1	4 read-only	Not registered yet
Shared	Grounded runs	context.retrieve.v1, added by AI-mode to runs of features with a registered corpus	Read-only	The feature's own corpus

5.2 RAG knowledge sources and grounding

Each feature registers one corpus of project-written guidance explaining its data, screens and limits. Corpora contain no live property facts, which come from MCP tools. Figure 3 shows ingestion and retrieval. A passage must score at least 0.55 to be used. AI-mode derives confidence from the evidence: gaps, a single source or a weak match give moderate; high needs two independent sources and strong matches. Appendix B lists every document.

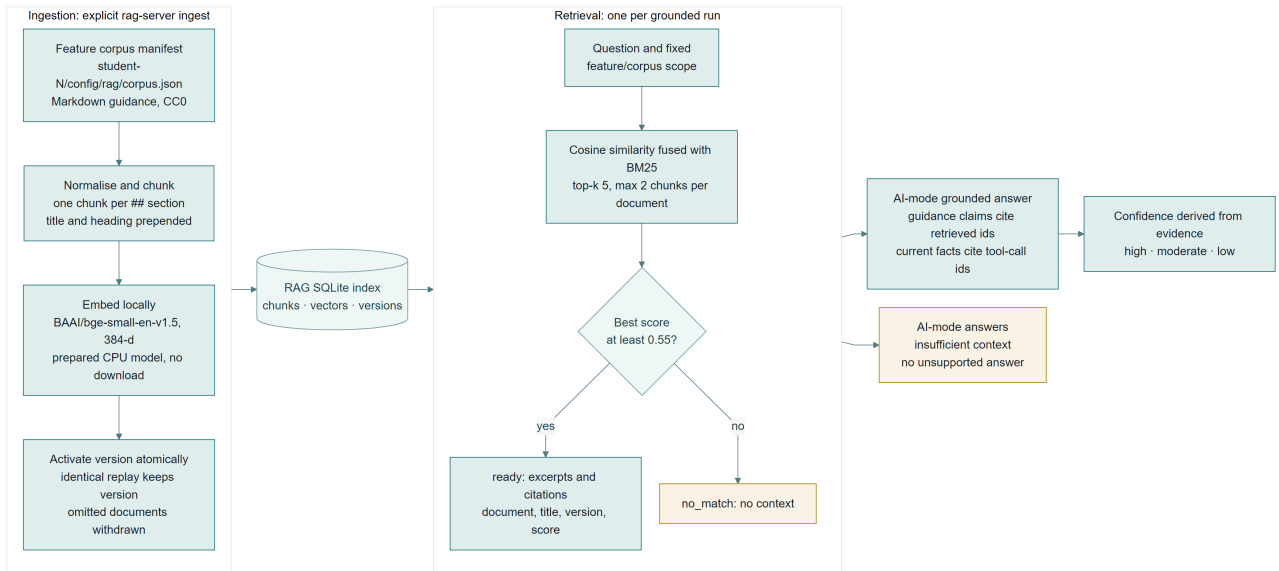


Figure 3 RAG ingestion, retrieval and grounded-answer rules

Feature 2's corpus covers what Sales and market is for and where its sale records come from, how a sale is attributed to a property and how confident that attribution is, and what recorded sales cannot establish, including that they are not a valuation and that fixture cases carry synthetic showcase figures.

TO COMPLETE / STUDENTS 3-5

Register your corpus (docs/release-1/adopt-mcp-and-rag.md steps 1-3). The table in 5.1 and Appendix B update automatically; add one sentence here on what your corpus covers.

6 Validation and results

We validated at three levels: deterministic tests in CI (MCP and RAG disabled), the shared loop's validation modes against live local services, and questions asked through each feature's frontend on the running stack. Screenshots and loop output below come from the live stack unless marked.

6.1 Terminal validation of the MCP and RAG servers

IMAGE PENDING

Figure 22 Activity history listing recent runs across features
docs/reports/assets/release-1/screenshots/shared-activity-history.png

TO COMPLETE / SHARED

With the stack up in combined mode, paste `uv run scripts/dev.py ai status` and `uv run scripts/dev.py ai probe` output at the submission commit as a text block here. The probe checks each server's authentication, the MCP tools/list per feature, each corpus version, a grounded retrieval and an insufficient-context query.

6.2 Shared agentic loop: MCP and RAG validation modes

Both modes drive the production runner against the live MCP and RAG servers with deterministic model decisions, so they test the integration rather than answer quality (Figure 4).

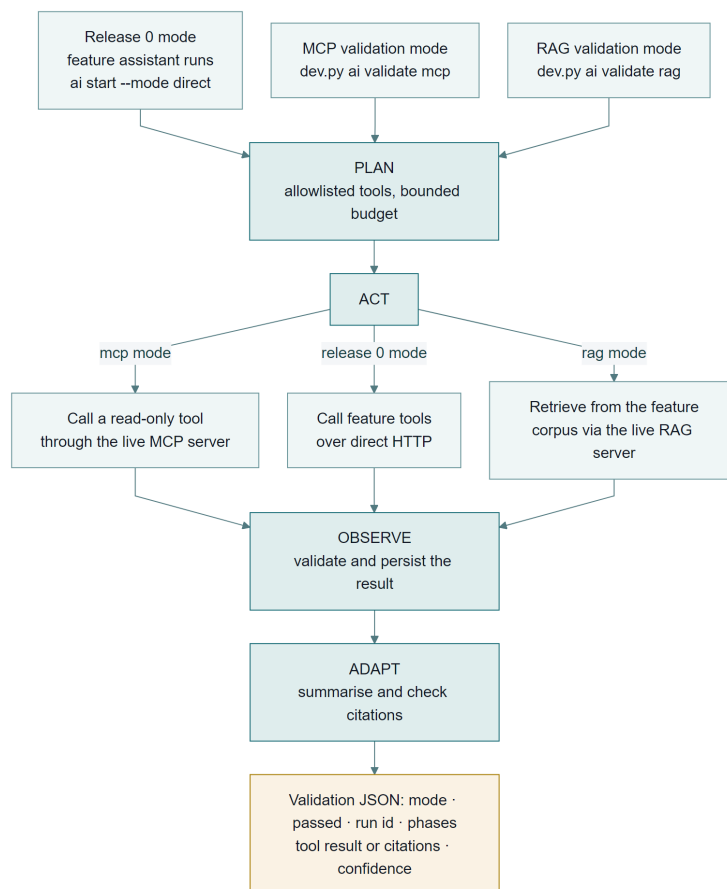


Figure 4 Shared agentic loop entry points and validation modes

```

$ uv run scripts/dev.py ai validate mcp
mode      mcp  passed=true  status=succeeded
run_id    56776025-5b0f-4388-978e-f89e850b6137
  
```

```
phases      plan -> act -> observe -> adapt
transport   live-local-services  decisions=deterministic-validation
tool call   succeeded in 16 ms; structured fields: application, assistant, feature_status_meaning,
features
summary     Registered MCP tool returned structured evidence.

$ uv run scripts/dev.py ai validate rag
mode        rag  passed=true  status=succeeded
run_id      c94635e7-2a85-40fc-b693-e71ad44cecc7
phases      plan -> act -> observe -> adapt
transport   live-local-services  decisions=deterministic-validation
tool call   succeeded in 93 ms, retrieval ready, 5 citations
  cites     Release provenance and artifact verification
  cites     Operator guidance, freshness and trust
  cites     Producer publication and downstream import
final       confidence=moderate  grounding=ready
summary     Retrieved project guidance.
```

TO COMPLETE / SHARED

These captures are from 7 September. Re-run `dev.py ai validate mcp --output docs/release-1/evidence/validation-mcp.json` and the rag equivalent at the submission commit, and optionally once per feature with `--feature`.

6.3 Feature validation through the frontend

Feature	Frontend, API and database (Release 0)	AI-mode	MCP result in UI	Grounded answer	Insufficient context
1 Property data	Source CRUD and property search, browser suite passes	Yes	Yes	Yes	Yes
2 Market intelligence	Case CRUD, sales evidence and deterministic summaries unchanged	Yes	Yes	Yes	Yes
3 Suburb analytics	Pending	Pending	Pending	Pending	Pending
4 Due diligence	Pending	Pending	Pending	Pending	Pending
5 Buyer workspace	Pending	Pending	Pending	Pending	Pending

TO COMPLETE / STUDENTS 3-5

Replace "Pending" in your row once each item is observed on the live stack, and add your three screenshots below with `capture_release1_screenshots.py`.

Feature 1 / Property data

IMAGE PENDING

Figure 5 Feature 1 MCP tool result shown in the assistant
`docs/reports/assets/release-1/screenshots/feature-1-mcp.png`

IMAGE PENDING

Figure 6 Feature 1 grounded answer with citations and confidence
`docs/reports/assets/release-1/screenshots/feature-1-rag.png`

IMAGE PENDING

Figure 7 Feature 1 insufficient-context answer
docs/reports/assets/release-1/screenshots/feature-1-insufficient.png

Feature 2 / Market intelligence

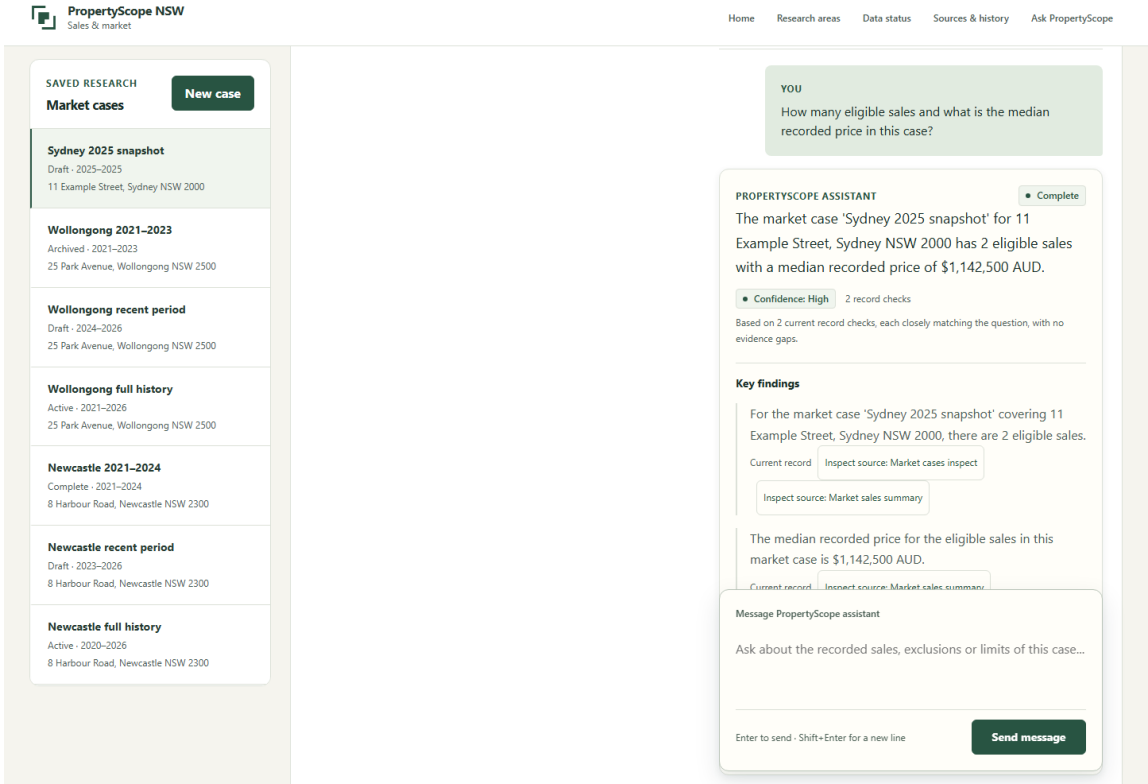


Figure 8 Feature 2 MCP tool result

IMAGE PENDING

Figure 9 Feature 2 grounded answer
docs/reports/assets/release-1/screenshots/feature-2-rag.png

IMAGE PENDING

Figure 10 Feature 2 insufficient-context answer
docs/reports/assets/release-1/screenshots/feature-2-insufficient.png

Feature 3 / Suburb analytics

IMAGE PENDING

Figure 11 Feature 3 MCP tool result
docs/reports/assets/release-1/screenshots/feature-3-mcp.png

IMAGE PENDING

Figure 12 Feature 3 grounded answer
docs/reports/assets/release-1/screenshots/feature-3-rag.png

IMAGE PENDING

Figure 13 Feature 3 insufficient-context answer
docs/reports/assets/release-1/screenshots/feature-3-insufficient.png

Feature 4 / Due diligence

IMAGE PENDING

Figure 14 Feature 4 MCP tool result
docs/reports/assets/release-1/screenshots/feature-4-mcp.png

IMAGE PENDING

Figure 15 Feature 4 grounded answer
docs/reports/assets/release-1/screenshots/feature-4-rag.png

IMAGE PENDING

Figure 16 Feature 4 insufficient-context answer
docs/reports/assets/release-1/screenshots/feature-4-insufficient.png

Feature 5 / Buyer workspace

IMAGE PENDING

Figure 17 Feature 5 MCP tool result
docs/reports/assets/release-1/screenshots/feature-5-mcp.png

IMAGE PENDING

Figure 18 Feature 5 grounded answer
docs/reports/assets/release-1/screenshots/feature-5-rag.png

IMAGE PENDING

Figure 19 Feature 5 insufficient-context answer
docs/reports/assets/release-1/screenshots/feature-5-insufficient.png

6.4 Retrieval evidence

Feature 1's retrieval baseline uses a disposable index and the same embedding model as the live server. Recall counts how many of each question's expected documents appear in the top five.

Measure	Result
Corpus	19 documents, 46 chunks, version da29fbf773df
Embedding	BAAI/bge-small-en-v1.5, 384 dimensions, local CPU
Settings	top-k 5, relevance floor 0.55
Supported questions	27; expected-source recall@5 0.963
Questions the corpus does not cover	4 of 6 retrieve no context
Evaluated	2026-09-26

The Knowledge sources page lists each registered corpus and its passages, and tests which passages a question retrieves.

IMAGE PENDING

Figure 21 Knowledge sources page with the registered corpora
docs/reports/assets/release-1/screenshots/shared-knowledge-sources.png

Retrieval similarity is not proof that an answer is correct. A share-price question still retrieves a sales passage (0.58), so the model and the confidence rules must still reject it.

6.5 GitHub Actions with MCP and RAG disabled

Every workflow sets `AI_MODE_MCP_ENABLED=false` and `AI_MODE_RAG_ENABLED=false` and uses test doubles for MCP and RAG contracts. They build the feature images, start the feature stack and smoke the frontend, API and database (Figure 20).

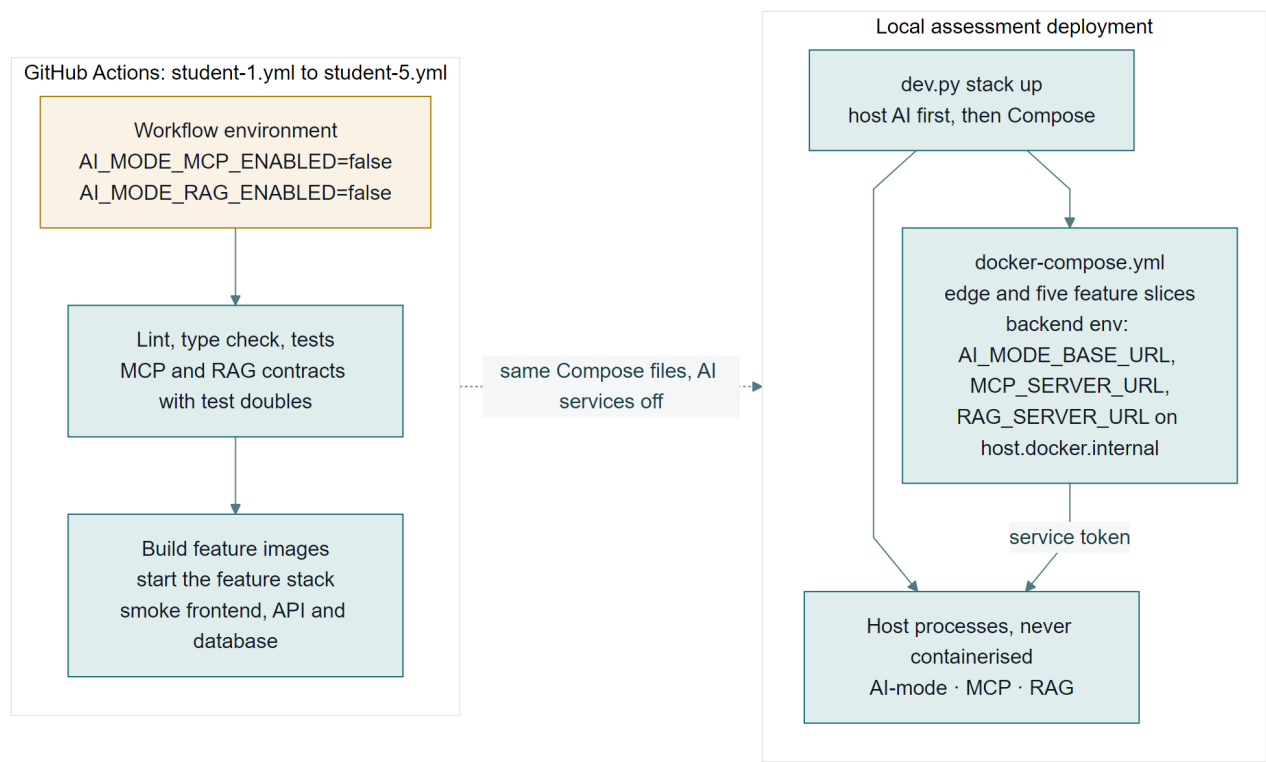


Figure 20 CI with AI services disabled, and the local host deployment

Workflow	Commit	Run	Result
Integration CI	0691e0f	36293980711	Success
Student 1 CI	0691e0f	36293980721	Success
Student 2 CI	0691e0f	36293980748	Success
Student 3 CI	0691e0f	36293980728	Success
Student 4 CI	0691e0f	36293980700	Success
Student 5 CI	0691e0f	36293980766	Success

TO COMPLETE / TEAM

Replace these 27 September runs with successful runs at the submission commit (`gh run list --limit 12`), after each student's Release 1 changes have merged.

6.6 Docker Compose deployment

All five backends receive `AI_MODE_BASE_URL` (Feature 3: `AI_MODE_URL`), `MCP_SERVER_URL` and `RAG_SERVER_URL` pointing at `host.docker.internal`. Compose defines no AI-mode, MCP, RAG or loop service.

TO COMPLETE / SHARED

Run `uv run scripts/dev.py stack up` at the submission commit and paste the `stack status` table (service, state, health), `ai status`, and HTTP 200 checks for the shared home and all five feature pages as text blocks here.

7 Integration summary

TO COMPLETE / TEAM

Three to five sentences once all features are validated: one stack started, every feature's Release 0 CRUD and AI-mode still work, and each feature's MCP and RAG interactions succeeded through its frontend. Link Section 6.3 rather than repeating it.

8 Known issues and limitations

Issue or limitation	Effect
AI-mode, MCP and RAG run on the developer's machine	The demonstration needs one prepared host with the embedding model and a model credential; there is no hosted instance until Release 2
Corpora are small, project-written guidance	They explain features and limits; they are not official methodology or live data
Retrieval can rank related but irrelevant text above the floor	Insufficient-context handling then relies on the model and confidence rules, not retrieval alone
Loop validation modes use deterministic model decisions	They prove the integration path, not answer quality; live answers are shown separately
Confidence is an evidence-support category	It is not a probability and does not verify that each claim follows from its source
Some demonstration data is labelled fixture data	Screenshots show fixture records where official data is not loaded
Feature 2 answers one selected market case	The assistant is withheld until a case is selected, and it cannot compare cases or answer without one

TO COMPLETE / STUDENTS 3-5

Add any limitation specific to your feature (one row each, if any).

9 Contributions, repository and showcase

Repository: github.com/MattShelton04/41026ASDProject. Detailed contribution logs and Release 1 commits are in Appendix C.

Student	Release 1 contribution	Pull requests
Matthew	Shared MCP server, RAG server, host runtime, loop validation modes, grounding and confidence rules, adoption kit, Feature 1 corpus and grounded assistant	#107 , #116 , #117 and others in Appendix C
Burhan	Feature 2 shared MCP and RAG adoption: grounded allowlist ownership, operator-guidance corpus, argument-free capability tool, shared assistant UI with citations and confidence	#120 and Appendix C
James	Pending	Pending
Michael	Pending	Pending
Derek	Pending	Pending

TO COMPLETE / STUDENTS 3-5

Fill in your row with your merged Release 1 pull requests, and add your log to Appendix C.

TO COMPLETE / TEAM

Add the showcase video URL here and record Week 9 attendance for all five students.

Appendix A Registered tool inputs and outputs

Generated from each feature's `tool-catalog.yaml` at build time.

Tool	Required inputs	Output fields	Side effect	Timeout
platform.capabilities.v1	None	revision, application, assistant, features, suggested_questions, feature_status_meaning	read-only	3000 ms
data.sources.v1	None	items, count	read-only	3000 ms
data.releases.v1	None	items, count	read-only	3000 ms
data.runs.v1	None	items, count	read-only	3000 ms
data.release_inspect.v1	release_id	release, quality_results, quality_summary, receipts, activations, consumer_imports, publication_policy, accepted_predecessor, release_contract	read-only	3000 ms
data.run_inspect.v1	run_id	run, tasks, quality_results	read-only	3000 ms
data.run_explain.v1	run_id	run, tasks, quality_results, insight	read-only	3000 ms
data.release_compare.v1	candidate_release_id, predecessor_release_id	candidate, predecessor, differences	read-only	3000 ms
data.coverage.v1	locality	items	read-only	3000 ms
property.search.v1	query	items, count	read-only	3000 ms
property.inspect.v1	property_ref	property, identifiers, aliases, coverage, sales_history	read-only	3000 ms
property.locality_summary.v1	None	availability, scope, total_registered_addresses, unit_number_summary, bounding_box, top_streets, release	read-only	5000 ms
data.run_retry.v1	run_id, idempotency_key	child_run_id, replayed	reversible-write, approval	5000 ms
data.release_publish.v1	release_id, idempotency_key	status, receipt_id, replayed	destructive-write, approval	5000 ms
market.capabilities.v1	None	revision, feature, tools, limitations, suggested_questions	read-only	3000 ms
market.cases.inspect.v1	market_case_id	market_case	read-only	3000 ms
market.sales.summary.v1	market_case_id	summary, sales	read-only	3000 ms
suburb.snapshot.v1	locality	suburb, places, limitations	read-only	3000 ms

Tool	Required inputs	Output fields	Side effect	Timeout
crime.compare.v1	localities, from_month, to_month, measure	series, measure, offence, source_release, limitations	read-only	4000 ms
crime.methodology.v1	None	geography, measures, offence_categories, zero_missing_rule, rate_method, responsible_use, source_release	read-only	2000 ms
duediligence.review.inspect.v1	site_review_id	site_review	read-only	3000 ms
duediligence.evidence.summary.v1	site_review_id	constraints, buildings	read-only	3000 ms
buyer.cases.inspect.v1	buyer_case_id	buyer_case, shortlisted_properties, property_count, limit, content_is_untrusted	read-only	5000 ms
buyer.notes.list.v1	buyer_case_id	items, count, limit, content_is_untrusted	read-only	5000 ms
buyer.tasks.list.v1	buyer_case_id	items, count, limit, content_is_untrusted	read-only	5000 ms
buyer.evidence.collect.v1	buyer_case_id	state, sections, evidence_references, limitations, bounds, content_is_untrusted	read-only	10000 ms

Appendix B Knowledge sources

Generated from each registered config/rag/corpus.json. All documents are project-written guidance published under CC0.

Feature	Document	Title	Source date
1	accepted-discovery	Searching for an address	2026-09-26
1	consumer-boundary	How other research areas receive data	2026-09-26
1	crime-coverage	Crime data coverage	2026-09-26
1	data-updates	Running a data update	2026-09-26
1	datasets	Datasets in Property data	2026-09-26
1	glossary	Property data terms	2026-09-26
1	gnaf-addresses	NSW address records (G-NAF)	2026-09-26
1	guidance-limits	How the assistant uses this guidance	2026-09-26
1	import-diagnosis	Diagnosing a failed data update	2026-09-26

Feature	Document	Title	Source date
1	missing-evidence	Missing evidence is unknown, not zero	2026-09-26
1	partial-psi	Selected PSI archive years are partial	2026-09-26
1	property-page	Reading a property record	2026-09-26
1	provenance	Where a value came from	2026-09-26
1	publication	Publishing a version	2026-09-26
1	recovery	Retrying publication and downstream imports	2026-09-26
1	reference-layers	Planning, hazard and boundary layers	2026-09-26
1	sales-history	NSW property sales (PSI)	2026-09-26
1	schools	School locations and catchments	2026-09-26
1	seifa	ABS SEIFA 2021 area scores	2026-09-26
2	what-this-feature-does	What Sales and market does and does not claim	2026-09-28
2	evidence-limits	What Sales and market evidence cannot establish	2026-09-28
2	match-quality	How sales are attributed to a property and how confident that is	2026-09-28

Appendix C Contribution logs and Release 1 commits

Student 1 Matthew Shelton

Date	Commit	Work
6 September	fa08c61	Release 1 plan and requirements analysis
7 September	1e109ac	Shared MCP and RAG runtime, loop validation modes, Feature 1 grounding
8 September	4aee88a	Host runtime safety, shutdown and setup guidance
13 September	d1e28ec	Shared assistant canvas and evidence history
19 September	9bc4ee9	Self-service MCP/RAG adoption kit for feature owners
26 September	e7485af	Retrieval calibration, relevance floor, evidence-derived confidence, corpus inspection page
27 September	0691e0f	Agent instructions, skills and dev . py improvements

Student 2 Burhan Naeem

Date	Commit	Work
28 September	a1b8577	Approved the grounded tool allowlist so Feature 2 can read its own grounded runs, with regression tests for the pre-grounding, grounded and foreign-allowlist cases
28 September	a1b8577	Wrote the three operator-guidance corpus documents, registered the corpus in student - 2/feature.yaml and ingested it
28 September	a1b8577	Added the argument-free market.capabilities.v1 tool so the shared loop has a read-only tool to call in MCP validation mode
28 September	a1b8577	Replaced the answer-text-only renderer with the shared assistant, accepted the shared {message, scope, context, history} payload and bounded assistant history
28 September	a1b8577	Captured MCP and RAG validation evidence under student - 2/docs/release - 1/evidence/

Student 3 James Huang

TO COMPLETE / STUDENT 3

Contribution log (date, commit, work) for Release 1.

Student 4 Michael White

TO COMPLETE / STUDENT 4

Contribution log (date, commit, work) for Release 1.

Student 5 Derek Song

TO COMPLETE / STUDENT 5

Contribution log (date, commit, work) for Release 1.

Appendix D Evidence index and reproduction

- [ADR-043 local grounded runtime](#)
- [ADR-046 non-containerised AI tier](#)
- [Host runtime guide](#)
- [Adopt MCP and RAG checklist](#)
- [Retrieval evaluation](#)
- [Captured validation outputs](#)
- [Shared and Feature 1 handoff](#)
- [Docker Compose model](#)
- [Student workflows](#)

```
uv sync --locked --all-packages --all-groups
uv run python scripts/check.py
uv run scripts/dev.py stack up
uv run scripts/dev.py ai status
uv run scripts/dev.py ai probe
uv run scripts/dev.py ai validate mcp
uv run scripts/dev.py ai validate rag
uv run python scripts/build_release1_report.py
```